

АНАЛИЗ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

Индивидуальное задание 29-го варианта.

Задан массив из 100 символов, который может содержать только буквы латинского алфавита.

Найти самые длинные последовательности гласных букв и отсортировать их между собой по длине, не нарушая самой последовательности. Согласные буквы отсортировать между собой по возрастанию. Применить сортировку отбором.

Объявить одномерный статический массив заданного типа и размера, заполнить его одним из способов по выбору пользователя: пользовательскими данными с клавиатуры, либо случайными числами в диапазоне от А до В (значения А и В ввести с клавиатуры).

Преобразовать массив в соответствии с индивидуальным заданием. Организовать вывод исходного массива и массива после обработки.

Написать программный код реализации составленного алгоритма с учетом требований и ограничений по индивидуальному заданию.

Организовать текстовый пользовательский интерфейс в программе.

Ввод данных пользователем реализовать с клавиатуры, а вывод результатов выполнения программы в консоль (на экран).

При выполнении задания следует учесть, что правильность вводимых значений не гарантируется. Необходимо обеспечить проверку данных и предусмотреть сообщение об ошибке в случае ее обнаружения.

Проверить правильность вычислений на тестовых примерах, выполнив серию контрольных прогонов программы.

Построить укрупненную схему составленного алгоритма (блок-схему работы программы).

ВЫПОЛНЕНИЕ ЗАДАНИЯ

```
#include <iostream>
#include <algorithm>
#include <cstring>
#include <Windows.h>
using namespace std;

// Функция для рандомизации целого числа в диапазоне
int randInt(int start, int end) { return rand() % (end - start + 1) + start; }

// Функция для проверки, является ли символ гласной
bool isVowel(char c)
{
    c = toupper(c);
    return (c == 'A' || c == 'E' || c == 'I' || c == 'O' || c == 'U' || c == 'Y'); //
    Символы 'a', 'e', 'i', 'o', 'u', 'y' являются гласными
}

// Функция сортировки отбором по убыванию длины строк
void selectionSortDescendingLength(string* arr, int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        int maxIdx = i;
        for (int j = i + 1; j < n; j++)
            if (arr[j].length() > arr[maxIdx].length())
                maxIdx = j;
        std::swap(arr[i], arr[maxIdx]);
    }
}

// Функция сортировки отбором по возрастанию символов
void selectionSortAscending(char* arr, int n) {
    for (int i = 0; i < n - 1; i++)
    {
        int minIdx = i;
        for (int j = i + 1; j < n; j++)
            if (arr[j] < arr[minIdx])
                minIdx = j;
        std::swap(arr[i], arr[minIdx]);
    }
}
```

```

        //Задан массив из 100 символов, который может содержать только
буквы
        //латинского алфавита.
        //Найти самые длинные последовательности гласных букв и
отсортировать их
        //между собой по длине, не нарушая самой
последовательности.Согласные
        //буквы отсортировать между собой по возрастанию.
        //Применить сортировку отбором.
int main()
{
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);
    setlocale(LC_ALL, "Rus");

    const int n = 100;
    char letters[n];

    int menu;
    int ind = 0;
    cout << "Выберите пункт меню\n";
    cout << "1. Заполнить массив вручную\n";
    cout << "2. Сгенерировать массив\n";
    cin >> menu;
    switch (menu)
    {
    case 1:
        cout << "Введите массив символов\n";
        while (ind < n)
        {
            cout << "Введите " << ind + 1 << "-й элемент: ";
            cin >> letters[ind];
            if (letters[ind] < 65 || letters[ind] > 90)
            {
                cout << "Некорректный символ. Символы
должны быть в верхнем регистре принадлежать латинскому алфвиту.\n";
                continue;
            }
            ind++;
        }
        break;
    default:
        cout << "Массив до сортировки: ";

```

```

        for (int i = 0; i < n; i++)
        {
            letters[i] = randInt(65, 90); // 65 - 90 -- диапазон
латинских символов в кодировке UTF-8
            cout << letters[i];
        }
        break;
    }
    cout << endl;

    string vowels[100];
    char consonants[100];
    int vowels_count = 0;
    int consonants_count = 0;

    for (int i = 0; i < n; i++) {
        if (isVowel(letters[i])) {
            int newI = i;
            vowels[vowels_count] = "";
            while (letters[newI] < n && isVowel(letters[newI]))
            {
                vowels[vowels_count] += letters[newI];
                newI++;
            }
            vowels_count++;
            i = newI;
        }
        else {
            consonants[consonants_count] = letters[i];
            consonants_count++;
        }
    }
    cout << endl;

    selectionSortDescendingLength(vowels, vowels_count);
    selectionSortAscending(consonants, consonants_count);

    cout << "Самые длинные последовательности гласных букв: ";
    for (int i = 0; i < vowels_count; i++) {
        cout << vowels[i] << "|";
    }
    cout << endl;

    cout << "Сортированные согласные буквы: ";

```

```

    for (int i = 0; i < consonants_count; i++) {
        cout << consonants[i];
    }
    cout << endl;
    return 0;
}

```

Результат выполнения программы (снимок экрана):

The screenshot shows the Microsoft Visual Studio IDE with a C++ project named 'OAIP LR 2'. The code in 'OAIP LR 2.cpp' implements a selection sort algorithm for sorting a string of letters. The console output shows the sorted string and the longest sequence of vowels.

```

// Функция для рандомизации целого числа в диапазоне
int randInt(int start, int end) { return rand() % (end - start + 1) + start; }

// Функция для проверки, является ли символ гласной
bool isVowel(char c)
{
    c = toupper(c);
    return (c == 'A' || c == 'E' || c == 'I' || c == 'O' || c == 'U' || c == 'Y'); // Символы 'a', 'e', 'i', 'o', 'u', 'y' являются гласными
}

// Функция сортировки отбором по убыванию длины строк
void selectionSortDescendingLength(string* arr, int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        int maxIdx = i;
        for (int j = i + 1; j < n; j++)
            if (arr[j].length() > arr[maxIdx].length())
                maxIdx = j;
        std::swap(arr[i], arr[maxIdx]);
    }
}

// Функция сортировки отбором по возрастанию символов
void selectionSortAscending(char* arr, int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        int minIdx = i;
        for (int j = i + 1; j < n; j++)
            if (arr[j] < arr[minIdx])
                minIdx = j;
        std::swap(arr[i], arr[minIdx]);
    }
}

int main()
{
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);
    setlocale(LC_ALL, "Rus");

    const int n = 100;
    char letters[n];

    // ... (rest of the code)
}

```

Консоль отладки Microsoft Visual Studio:

```

Выберите пункт меню
1. Заполнить массив вручную
2. Сгенерировать массив
2
Массив до сортировки: PHQGHUMEAYLNLFDXFIRCVSCXGGBHFNQDUXWFNFOZVSRKJPREPGGXRPNRVYSTHWCYSYYCQPEVIKEFFMZ
NIMKKASVWSRENZKYCXF
Самые длинные последовательности гласных букв: EAY|VY|I|U|O|E|V|Y|U|E|I|E|I|A|E|Y|
Сортированные согласные буквы: BCCDDFFFFFFFGGGGGHJKKKKLLMNNNNNNPPPPQQRRRRSSSTTVVVVWMMMMXXXXZZ
C:\Users\minsk\source\repos\OAIP LR 2\Debug\OAIP LR 2.exe (процесс 11776) завершил работу с кодом 0
Чтобы автоматически закрывать консоль при остановке отладки, включите параметр "Сервис" -> "Параметры" -
-> "Отладка" -> "Автоматически закрыть консоль при остановке отладки".
Нажмите любую клавишу, чтобы закрыть это окно.

```

Блок-схема алгоритма изображена на рисунке 2.1

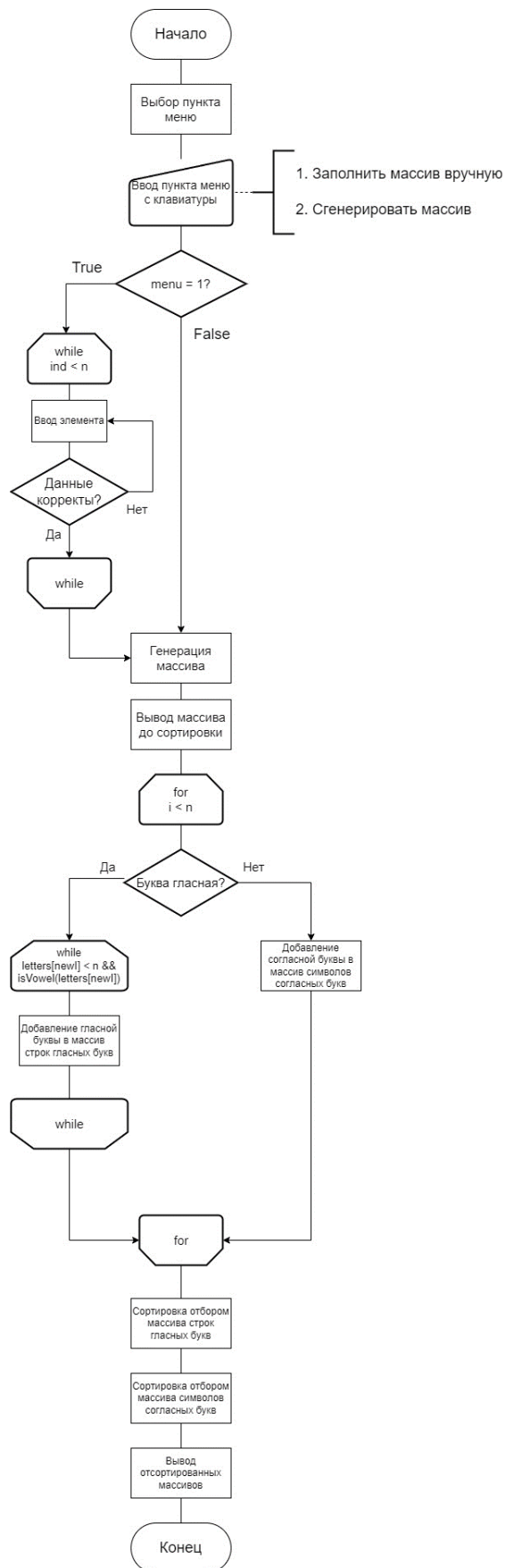


Рисунок 2.1 – Блок-схема алгоритма

ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Составные типы данных. Виды и их назначение.

К составным типам относятся массивы, перечисления, функции, структуры, ссылки, указатели, объединения и классы.

2. Особенности объявления массивов на C++.

```
int arr[50];
```

```
float myTestArrayForLesson[10 * 1000 + 1];
```

```
int arr[] = {0, 1, 2, 3, 4, 5}; // массив будет иметь 6 элементов (от 0 до 5)
```

```
int mas[100] = {0}; // все 100 элементов будут иметь значение 0
```

3. Инициализация массива. Возможные способы инициализации массивов в C++.

```
int a[4] = { 1, 2, 3, 4 };
```

```
int a[] { 1, 2, 3, 4 };
```

```
int a[4]{};
```

```
const int a[4] = { 3, 2, 1 };
```

```
const char str[] = "meow"; // Символьные массивы можно  
инициализировать строковым литералом
```

```
const wchar_t wstr[] = L"meow";
```

4. Доступ к элементам массива. Особенности обработки массивов на языке C++.

```
int marks[5];
```

```
marks[0] = 19;
```

Для передачи массива в функцию в качестве аргумента нужно передавать не сам массив, а указатель на начало массива.

5. Основные операции, допустимые с массивами на языке C++.

Узнать количество элементов в массиве.

Перебор элементов массива в цикле.

Вычисление суммы элементов массива

Поиск максимального элемента в массиве

Фильтрация элементов массива

Сортировка элементов в массиве

Удаление элемента из массива

6. Общая формулировка алгоритма сортировки данных.

Алгоритм сортировки — это алгоритм для упорядочивания элементов в списке.

7. Разновидности, характеристики и особенности различных алгоритмов сортировки.

Сортировка классифицируется по устойчивости, естественности поведения, использования операции сравнения и сфере применения. Устойчивость (стабильность) — устойчивая сортировка не меняет взаимного расположения равных элементов.

8. Суть метода «пузырьковой сортировки».

Меняет местами соседние элементы, пока весь массив не будет отсортирован.

9. Суть метода «сортировки отбором».

Ищет наименьший элемент и перемещает его в начало массива, затем повторяет процесс для оставшейся части массива.

10. Суть метода «сортировки вставками».

Поочередно вставляет элементы в уже отсортированную часть массива, пока весь массив не будет отсортирован.

11. Суть метода «быстрой сортировки».

Выбирает опорный элемент, разделяет массив на две части (меньшие и большие элементы относительно опорного) и рекурсивно сортирует каждую часть.

12. Понятие «скорость сортировки». От чего зависит скорость сортировки для разных алгоритмов.

Скорость сортировки — трудоемкость алгоритма. Трудоемкость алгоритма пропорциональна функции $n * \log(n)$.

То есть, при некотором количестве процессорных инструкций при количестве элементов массива равном n , можно замерить время выполнения при $n = 10$, $n = 100$, и спрогнозировать время выполнения при $n = 1000000$.

13. Общая формулировка алгоритма поиска данных в линейных структурах. Разновидности алгоритмов поиска.

Поиск — шаблон для поиска элемента, применимый к различным типам линейных структур данных, таким как массивы, списки или очереди. Этот алгоритм позволяет последовательно проверять каждый элемент структуры на соответствие искомому значению и возвращать результат, когда элемент будет найден или цикл будет завершен.

14. Суть метода «линейного поиска». Особенности реализации и характеристики.

Линейный поиск определяется как последовательный алгоритм поиска, который начинается с одного конца и проходит по каждому элементу

списка, пока не будет найден нужный элемент, в противном случае поиск продолжается до конца набора данных.

15. Суть метода «двоичного поиска». Особенности реализации и характеристики.

Двоичный (бинарный) поиск — классический алгоритм поиска элемента в отсортированном массиве (векторе), использующий дробление массива на половины.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были изучены теоретические сведения, составлена блок-схема индивидуального задания и реализовано программное средство, удовлетворяющее поставленной задаче, даны ответы на контрольные вопросы.